

Sistemas Concurrentes y Distribuidos: **Problemas Resueltos.**

Dpt. Lenguajes y Sistemas Informáticos
ETSI Informática y de Telecomunicación
Universidad de Granada

Curso 2024-25



Universidad de Granada

Índice general

1. Introducción	5
Problema 1.	5
Problema 2.	6
Problema 3.	7
Problema 4.	10
Problema 5.	11
Problema 6.	12
Problema 7.	14
Problema 8.	15
Problema 9.	16
Problema 10.	19
Problema 11.	20
Problema 12.	21
2. Sincronización en memoria compartida.	23
Problema 13.	23
Problema 14.	24
Problema 15.	25
Problema 16.	29
Problema 17.	30
Problema 18.	30
Problema 19.	31
Problema 20.	33
Problema 21.	34
Problema 22.	36
Problema 23.	38
Problema 24.	43
Problema 25.	46
Problema 26.	48
Problema 27.	51

Problema 28.	52
Problema 29.	54
Problema 30.	55
3. Sistemas basados en paso de mensajes.	59
Problema 31.	59
Problema 32.	59
Problema 33.	60
Problema 34.	61
Problema 35.	63
Problema 36.	65
Problema 37.	66
Problema 38.	67
Problema 39.	69
Problema 40.	70
Problema 41.	71
Problema 42.	73
Problema 43.	74
Problema 44.	75
Problema 45.	76
Problema 46.	76
Problema 47.	78
4. Sistemas de Tiempo Real.	81
Problema 48.	81
Problema 49.	82
Problema 50.	83
Problema 51.	84
Problema 52.	85
Problema 53.	85
Problema 54.	87

1

Considerar el siguiente fragmento de programa para 2 procesos P_1 y P_2 :

Los dos procesos pueden ejecutarse a cualquier velocidad. ¿Cuáles son los posibles valores resultantes para x ? Suponer que x debe ser cargada en un registro para incrementarse y que cada proceso usa un registro diferente para realizar el incremento.

```
{ variables compartidas }
var x : integer := 0 ;
```

```
process P1 ;
  var i : integer ;
begin
  for i := 1 to 2 do begin
    x := x+1 ;
  end
end
```

```
process P2 ;
  var j : integer ;
begin
  for j := 1 to 2 do begin
    x := x+1 ;
  end
end
```

Respuesta

Los valores posibles son 2, 3 y 4. Suponemos que no hay optimizaciones al compilar y que por tanto cada proceso hace dos lecturas y dos escrituras de x en memoria. La respuesta se basa en los siguientes tres hechos:

- el valor resultante no puede ser inferior a 2 pues cada proceso incrementa x dos veces en secuencia partiendo de cero, la primera vez que un proceso lee la variable lee un 0 como mínimo, y la primera vez que la escribe como mínimo 1, la segunda vez que ese mismo proceso lee, lee como mínimo un 1 y finalmente escribe como mínimo un 2.
- el valor resultante no puede ser superior a 4. Para ello sería necesario realizar un total de 5 o más incrementos de la variable, cosa que no ocurre pues se realizan únicamente 4.
- existen posibles secuencias de interfoliación que producen los valores 2,3 y 4, damos ejemplos de cada uno de los casos:

resultado 2: se produce cuando todas las lecturas y escrituras de un proceso i se ejecutan completamente entre la segunda lectura y la segunda escritura del otro proceso j . La segunda lectura de j lee un 1 y escribe un 2, siendo esta escritura la última en realizarse y por tanto la que determina el valor de x

resultado 3: se produce cuando los dos procesos leen y escriben x por primera vez de forma simultánea, quedando x a 1. Los otros dos incrementos se producen en secuencia (un proceso escribe antes de que lea el otro), lo cual deja la variable a 3.

resultado 4: se produce cuando un proceso hace la segunda escritura antes de que el otro haga su primera lectura. Es evidente que el valor resultado es 4 pues todos los incrementos se hacen secuencialmente.

2

Dada la declaración de proceso secuencial (que forma parte de un programa concurrente) que aparece abajo, donde S es una sentencia que puede ser simple o compuesta, indica, razonando la respuesta, qué versiones de S son sentencias atómicas y cuáles no lo son. Para aquellas sentencias no atómicas, dar la traza detallada indicando aquel estado o estados que sean accesibles por otros procesos secuenciales.

```
{ Declaración variables globales }
var x : integer := 0;
    z : integer := 2;

{ Procesos }
process P;
    var i : integer := 0;
        j : integer := 1;
begin
    S;
end
```

- (a) $S \equiv z := i + 1;$
- (b) $S \equiv z := 2 * x + j;$
- (c) $S \equiv i := z + j;$
- (d) $S \equiv i := i + j; x := i + 1;$
- (e) $S \equiv x := x + j;$
- (f) $S \equiv j := z + 2; i := i * 5 + j; j := j + 1;$
- (g) $S \equiv j := i + 2; i := j + 1; x := j + i;$
- (h) $S \equiv x := 2 * z + x;$
- (i) $S \equiv i := 2 * i + j + 3; j := j + i * i;$
- (j) $S \equiv j := z + 2 * x;$

Respuesta

Para las no atómicas, se dará la traza detallada abajo resaltando los estados intermedios accesibles.

Respecto a cuales son atómicas y cuales no, en estos casos ninguna instrucción tiene los ángulos $\langle$ y $\rangle$, por tanto, para decidir si son atómicas o no, basta con ver cuantos accesos hay a variables compartidas:

- (a) $S \equiv z := i + 1;$
Atómica, la sentencia únicamente realiza un acceso a variables compartidas (accede a la variable z para escribir). No hay estados intermedios accesibles a otros procesos.
- (b) $S \equiv z := 2 * x + j;$
No atómica. La sentencia realiza dos accesos a variables compartidas. Primero lee x y luego escribe z . Entre ambos accesos hay estados intermedios accesibles a otros procesos.
- (c) $S \equiv i := z + j;$
Atómica, la sentencia únicamente realiza un acceso a variables compartidas (accede a la variable z para leerla). No hay estados intermedios accesibles a otros procesos.
- (d) $S \equiv i := i + j; x := i + 1;$
Atómica, únicamente realiza un acceso de escritura en las variables compartidas (escribe en x al final). No hay estados intermedios accesibles a otros procesos.
- (e) $S \equiv x := x + j;$
No atómica. Realiza dos accesos a las variables compartidas, primero lee x y luego la escribe. Entre ambos accesos hay estados intermedios accesible a otros procesos.
- (f) $S \equiv j := z + 2; i := i * 5 + j; j := j + 1;$
Atómica, únicamente realiza un acceso a las variables compartidas (lee de z al inicio). No hay estados intermedios accesibles a otros procesos.
- (g) $S \equiv j := i + 2; i := j + 1; x := j + i;$
Atómica, únicamente realiza un acceso a las variables compartidas (escribe en x al final). No hay estados intermedios accesibles a otros procesos.
- (h) $S \equiv x := 2 * z + x;$
No atómica. Realiza dos accesos a las variables compartidas, primero lee x y luego la escribe. Entre ambos accesos hay un estado intermedio accesible a otros procesos.
- (i) $S \equiv i := 2 * i + j + 3; j := j + i * i;$
Atómica. No realiza accesos a variables compartidas. No hay estados intermedios accesibles a otros procesos.
- (j) $S \equiv j := z + 2 * x;$
No atómica. Realiza dos accesos a las variables compartidas, primero lee x y luego lee z . Entre ambos accesos hay un estado intermedio accesible a otros procesos.

3

Dado el programa concurrente que se muestra abajo, responde a cada una de estas cuestiones:

- (a) indica cómo se descomponen las sentencias de cada proceso en secuencias de sentencias que únicamente hacen operaciones aritméticas en los registros,
- (b) indica como se descomponen las sentencias de cada proceso en un número mínimo de sentencias atómicas,

- (c) proporciona todos los posibles valores que puede tomar la variable x al finalizar la ejecución del programa,
- (d) proporciona una traza (en base a las sentencias del caso (a)) para cada valor diferente de x que se pueda obtener al finalizar la ejecución concurrente,
- (e) indica cuantas interfoliaciones posibles hay de las sentencias del caso (a) y cuantas del caso (b).

```
{ Variables compartidas }
var x : integer := 0 ;
```

```
process P0;
  var i0 : integer := 3 ;
begin
  x := 2*i0+1;
end
```

```
process P1;
  var i1 : integer := 1 ;
begin
  x := x+i1;
end
```

Respuesta

Apartado (a): Indica cómo se descomponen las sentencias de cada proceso en secuencias de sentencias que únicamente hacen operaciones aritméticas en los registros.

Para descomponer las sentencias, tal y como se pide en este apartado, suponemos que el registro $r0$ estará asignado a $P0$ mientras que los registros $r10$ y $r11$ serán asignados a $P1$.

La descomposición en sentencias (usando registros para las operaciones aritméticas) de $x := 2*i0+1$ es:

```
r0 := i0 ;
x  := 2*r0+1 ;
```

La descomposición de $x := x+i1$ es:

```
r10 := x ;
r11 := i1 ;
x   := r10+r11 ;
```

Apartado (b): Indica como se descomponen las sentencias de cada proceso en un número mínimo de sentencias atómicas.

La descomposición en un número mínimo de sentencias atómicas de la sentencia de $x := 2*i0+1$ es la propia sentencia, ya que esta sentencia es atómica de partida (únicamente hace un acceso a variables compartidas, la escritura en x):

```
x := 2*i0+1;
```

La descomposición en un número mínimo de sentencias atómicas de la sentencia de $x := x+i1$ podría en principio ser la descomposición en tres sentencias que se ha dado en el apartado (a), pero en este caso, la segunda y tercera sentencias, ejecutadas en secuencia, forman una única sentencia atómica, ya que entre ambas únicamente producen un único acceso a x . Por tanto, la descomposición que se nos pide de $x := x+i1$ es:

```
r10 := x ;
x   := r10+i1 ;
```


Apartado (c): Proporciona todos los posibles valores que puede tomar la variable **x** al finalizar la ejecución del programa.

La variable **x** puede tener al final uno de estos valores: **8, 7 y 1**.

Apartado (d): Proporciona una traza (en base a las sentencias del caso (a)) para cada valor diferente de **x** que se pueda obtener al finalizar la ejecución concurrente.

Una traza que da lugar al valor 8 es cualquiera en la que la escritura de **x** en **P0** se haga antes de la lectura de **x** en **P1**. Un ejemplo es esta que aparece aquí, en ella **P0** acaba antes de que empiece **P1**:

P0	P1	x	i0	i1	r0	r10	r11
		0	3	1	-	-	-
r0 := i0		0	3	1	3	-	-
x := 2*r0+1		7	3	1	3	-	-
	r10 := x	7	3	1	3	7	-
	r11 := i1	7	3	1	3	7	1
	x := r10+r11	8	3	1	3	7	1

Una traza que da lugar al valor 7 es cualquiera en la que el proceso **P0** escriba en **x** después de que lo haga **P1**. Un ejemplo es esta de aquí (en ella **P1** acaba antes de que empiece **P0**):

P0	P1	x	i0	i1	r0	r10	r11
		0	3	1	-	-	-
	r10 := x	0	3	1	-	0	-
	r11 := i1	0	3	1	-	0	1
	x := r10+r11	1	3	1	-	0	1
r0 := i0		1	3	1	3	0	1
x := 2*r0+1		7	3	1	3	0	1

Una traza que da lugar al valor 1 es cualquiera en la que la escritura de **x** en **P0** ocurra entre los dos accesos a **x** que hace **P1**, luego la ejecución de las dos sentencias se solapa. Por ejemplo, sería esta:

P0	P1	x	i0	i1	r0	r10	r11
		0	3	1	-	-	-
	r10 := x	0	3	1	-	0	-
r0 := i0		0	3	1	3	0	-
x := 2*r0+1		7	3	1	3	0	-
	r11 := i1	7	3	1	3	0	1
	x := r10+r11	1	3	1	3	0	1

Apartado (d): Indica cuantas interfiliaciones posibles hay de las sentencias del caso (a) y cuantas del caso (b).

En el caso (a) una sentencia se descompone en otras 2, y la otra se descompone en otras 3, por tanto, el número de interfiliaciones posibles es el dado por el coeficiente binomial, particularizando para $n_1 = 2$ y $n_2 = 3$, es decir, el número de interfiliaciones es

$$\frac{(n_1 + n_2)!}{n_1! n_2!} = \frac{5!}{2! \cdot 3!} = \frac{5 \cdot 4 \cdot 3 \cdot 2}{2 \cdot (3 \cdot 2)} = 10$$

En el caso (b) una sentencia no se descompone, y la otra se descompone en otras 2, por tanto ahora $n_1 = 1$ y $n_2 = 2$, es decir, ese número es:

$$\frac{(n_1 + n_2)!}{n_1!n_2!} = \frac{3!}{2! \cdot 1!} = \frac{3 \cdot 2}{2} = 3$$

Como vemos, si consideramos un número mínimo de sentencias atómicas, el número de interfiliaciones se reduce bastante.

4

Consideramos un programa concurrente en el cual varios procesos comparten dos variables enteras x e y , y queremos que siempre se cumpla $x+y == 10$.

Apartado (a)

Dado un valor n cualquiera (puede ser una constante o una variable local), para asignarle n a la variable x (y actualizar adecuadamente y) todos los procesos usan **alguna** de estas tres sentencias alternativas (todas la misma):

```
{ Opción 1 }
begin
  x := n;
  y := 10-n;
end
```

```
{ Opción 2 }
begin
  <x:= n>;
  <y:= 10-n>;
end
```

```
{ Opción 3 }
begin
  < x:= n;
  y:= 10-n; >
end
```

Para cada opción: describe razonadamente si la opción es correcta. Se considera correcta una opción si, al ejecutarse la sentencia bloque en un estado previo en el cual se cumple $x+y==10$, se garantiza que siempre en el estado inmediatamente posterior se cumplirá $x+y==10$.

Apartado (b)

Para verificar que la suma de ambas variables siempre es 10, escribimos código que imprime dicha suma. Los procesos podrán ejecutar ese código en cualquier momento, de forma concurrente con los procesos que actualizan x e y . Para ello, todos los procesos usarán una variable local propia (a) y **alguna** de las siguientes sentencias alternativas (todas la misma):

```
{ Opción 1 }
begin
  a := x+y;
  print(a);
end
```

```
{ Opción 2 }
begin
  < a:= x+y >;
  print(a);
end
```

```
{ Opción 3 }
begin
  < a:= x+y;
  print(a); >
end
```

Para cada opción: describe razonadamente si la opción es correcta. Se considera correcta una opción si, al ejecutarse la sentencia bloque en un estado previo con $x+y==10$, se garantiza que siempre se imprimirá el valor 10.

Además, responde a esta pregunta: ¿ es atómica la sentencia **begin-end** de la opción 2 ?

Respuesta

Apartado (a)

Opción 1

Es **incorrecta** ya que se hacen dos escrituras separadas en **x** y en **y**, la sentencia bloque no es atómica. Supongamos que un proceso **P** escribe 6 y 4 en **x** e **y** respectivamente, a la vez que otro proceso **Q** quiere escribir 7 y 3. Si las dos escrituras de **Q** ocurren ambas entre las dos escrituras de **P**, entonces al final los valores de **x** e **y** serán 7 y 4, así que la suma no sería 1.

Opción 2

Es **incorrecta** por el mismo motivo que la anterior, ya que hemos hecho atómicas las sentencias de escritura, pero esas sentencias ya eran atómicas antes, no hemos cambiado nada.

Opción 3

Es **correcta**, ya que hemos puesto los ángulos con las dos asignaciones dentro, por tanto hemos convertido en atómica la sentencia, y ningún proceso puede leer ni escribir ni **x** ni **y** cuando un proceso está ejecutando este **begin-end**.

Apartado (b)

Opción 1

Es **incorrecta** ya que se hacen dos lecturas separadas de **x** y de **y**, la sentencia bloque no es atómica. Supongamos que un proceso **P** ejecuta el **begin-end** en un estado en el cual los valores de **x** e **y** son 6 y 4, respectivamente, a la vez que otro proceso **Q** escribe 7 y 3. Si las dos escrituras de **Q** ocurren ambas entre las dos lecturas de **P**, entonces **P** leerá 6 y 3, y el valor impreso no sería 10.

Opción 2

Es **correcta**, ya que si un proceso **P** la ejecuta, hará las dos lecturas de **x** y de **y** sin que haya posibilidad de que ningún otro proceso acceda a las variables **x** e **y** entre las dos lecturas.

Se pregunta si la sentencia bloque **begin-end** es atómica, y la respuesta es que sí lo es. Aunque el **print** esté fuera de los ángulos, ese **print** únicamente accede a la variable local **a**. Así que el estado intermedio entre la lectura de **x** y la de **y** no es accesible por otros procesos.

Opción 3

Es **correcta**, ya que es una instrucción atómica por definición, y de hecho es equivalente a la anterior, ya que incluir el **print** dentro de los ángulos no cambia nada respecto a la atomicidad.

5

¿Cómo se podría hacer la copia del fichero **f** en otro **g**, de forma concurrente, utilizando la instrucción concurrente **cobegin-coend**? . Para ello, suponer que:

- los archivos son secuencia de ítems de un tipo arbitrario **T**, y se encuentran ya abiertos para lectura (**f**) y escritura (**g**). Para leer un ítem de **f** se usa la llamada a función **leer(f)** y para saber si se han leído todos los ítems de **f**, se puede usar la llamada **fin(f)** que devuelve verdadero si ha habido al menos un intento de leer cuando ya no quedan datos. Para escribir un dato **x** en **g** se puede usar la llamada a procedimiento

```
escribir(g, x).
```

- El orden de los ítems escritos en **g** debe coincidir con el de **f**.
- Dos accesos a dos archivos distintos pueden solaparse en el tiempo.

Para ilustrar como se accede a los archivos, aquí se encuentra una versión secuencial del código que copia **f** sobre **g**:

```
process CopiaSecuencial ;
  var v : T ;
begin
  v := leer(f) ;    { lectura adelantada }
  while not fin(f) do
    begin
      escribir(g, v) ; { leer de la variable v y escribir en el archivo g }
      v := leer(f) ;   { leer del archivo f y escribir variable v }
    end
  end
end
```

Respuesta

Los ítems deben ser escritos en secuencia para conservar el orden, así que la lectura y la escritura puede hacerse en un bucle secuencial. Sin embargo, se puede solapar en el tiempo la escritura de un ítem leído y la lectura del siguiente, y por tanto en cada iteración se usará un **cobegin-coend** con la lectura solapada con la escritura.

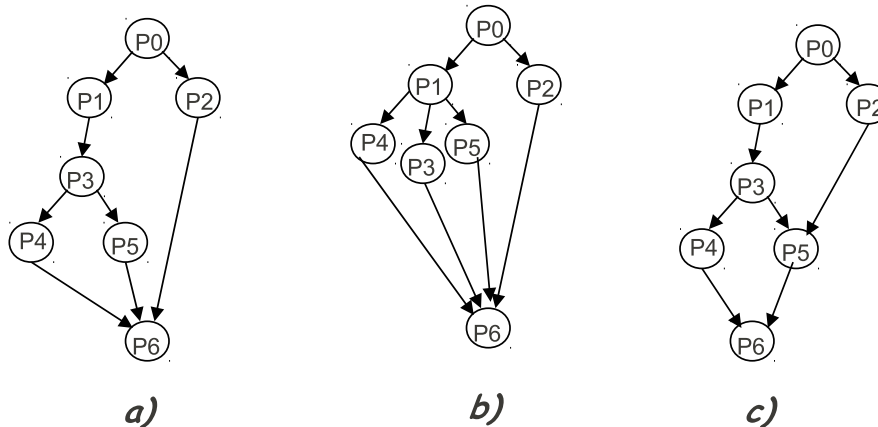
La solución más obvia sería usar una variable **v** (compartida entre la lectura y la escritura) para esto, es decir, usar en cada iteración la solución que aparece en la figura de la izquierda. El problema es que en esta solución la variable **v** puede ser accedida simultáneamente por la escritura y la lectura concurrentes, que podrían interferir entre ellas, así que es necesario usar dos variables. El esquema correcto quedaría como aparece en la figura de la derecha.

```
process CopiaConcurrenteMal ;
  var v : T ;
begin
  v := leer(f) ;
  while not fin(f) do
    cobegin
      escribir(g, v) ;
      v := leer(f) ;
    coend
  end
end
```

```
process CopiaConcurrente ;
  var v_ant, v_sig : T ;
begin
  v_sig := leer(f) ;
  while not fin(f) do begin
    v_ant := v_sig ;
    cobegin
      escribir(g, v_ant) ;
      v_sig := leer(f) ;
    coend
  end
end
```

6

Construir, utilizando las instrucciones concurrentes **cobegin-coend** y **fork-join**, programas concurrentes que se correspondan con los grafos de precedencia que se muestran a continuación:



Respuesta

A continuación incluimos, para cada grafo, las instrucciones concurrentes usando **cobegin-coend** (izquierda) y **fork-join** (derecha)

(a)

```
begin
  P0 ;
  cobegin
    begin
      P1 ; P3 ;
      cobegin
        P4 ; P5 ;
      coend
    end
    P2 ;
  coend
  P6 ;
end
```

```
begin
  P0 ; fork P2 ;
  P1 ; P3 ; fork P4 ; fork P5 ;
  join P2 ; join P4 ; join P5 ;
  P6 ;
end
```

(b)

```
begin
  P0 ;
  cobegin
    begin
      P1 ;
      cobegin
        P3 ; P4 ; P5 ;
      coend
    end
    P2 ;
  coend
  P6 ;
end
```

```
begin
  P0 ; fork P2 ;
  P1 ; fork P3 ; fork P4 ; fork P5 ;
  join P2 ; join P3 ;
  join P4 ; join P5 ;
  P6 ;
end
```

(c) en este caso, **cobegin-coend** no permite expresar el simultáneamente el paralelismo potencial que hay entre **P4** y **P2** y el que hay entre **P4** y **P5**, mientras **fork-join** sí permite expresar todos los paralelismos presentes (es más flexible).

```
begin
  P0 ;
  cobegin
    begin
      P1 ; P3 ;
    end
    P2 ;
  coend
  cobegin
    P4 ; P5 ;
  coend
  P6 ;
end
```

```
begin
  P0 ;
  cobegin
    begin
      P1 ; P3 ; P4 ;
    end
    P2 ;
  coend
  P5 ; P6 ;
end
```

```
begin
  P0 ; fork P2 ;
  P1 ;
  P3 ; fork P4 ;
  join P2 ;
  P5 ;
  join P4 ;
  P6 ;
end
```

7

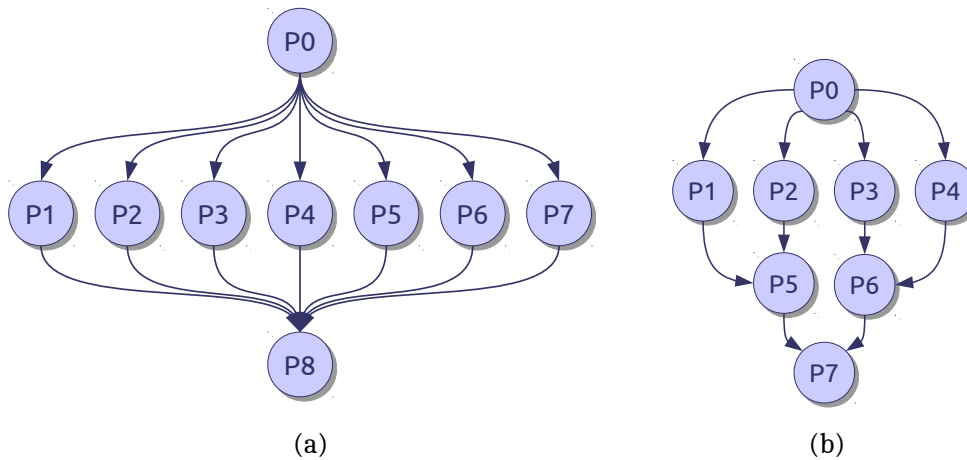
Dados los siguientes fragmentos de programas concurrentes, obtener sus grafos de precedencia asociados:

```
begin
  P0 ;
  cobegin
    P1 ;
    P2 ;
    cobegin
      P3 ; P4 ; P5 ; P6 ;
    coend
    P7 ;
  coend
  P8 ;
end
```

```
begin
  P0 ;
  cobegin
    begin
      cobegin
        P1;P2;
      coend
      P5;
    end
    begin
      cobegin
        P3;P4;
      coend
      P6;
    end
  coend
  P7 ;
end
```

Respuesta

En el caso a), anidar un bloque **cobegin-coend** dentro de otro, sin incluir ningún componente adicional en secuencia, tiene el mismo efecto que incluir directamente en el bloque externo las instrucciones del interno. Esta no es la situación en el caso b), donde las construcciones **cobegin-coend** anidadas son necesarias para reflejar ciertas dependencias entre actividades.



8

Suponer un sistema de tiempo real que dispone de un captador de impulsos conectado a un contador de energía eléctrica. La función del sistema consiste en contar el número de impulsos producidos en 1 hora (cada Kwh consumido se cuenta como un impulso) e imprimir este número en un dispositivo de salida.

Para ello se dispone de un programa concurrente con 2 procesos: un proceso acumulador (lleva la cuenta de los impulsos recibidos) y un proceso escritor (escribe en la impresora). En la variable común a los 2 procesos n se lleva la cuenta de los impulsos. El proceso acumulador puede invocar un procedimiento **Espera_impulso** para esperar a que llegue un impulso, y el proceso escritor puede llamar a **Espera_fin_hora** para esperar a que termine una hora.

El código de los procesos de este programa podría ser el siguiente:

```
{ variable compartida: }
var n : integer; { contabiliza impulsos }
```

```
process Acumulador ;
begin
  while true do begin
    Espera_impulso();
    < n := n+1 > ; { (1) }
  end
end
```

```
process Escritor ;
begin
  while true do begin
    Espera_fin_hora();
    write( n ) ; { (2) }
    < n := 0 > ; { (3) }
  end
end
```

En el programa se usan sentencias de acceso a la variable n encerradas entre los símbolos $< y >$. Esto significa que cada una de esas sentencias se ejecuta en exclusión mutua entre los dos procesos, es decir, esas sentencias se ejecutan de principio a fin sin entremezclarse entre ellas.

Supongamos que en un instante dado el acumulador está esperando un impulso, el escritor está esperando el fin de una hora, y la variable **n** vale k . Después se produce de forma simultánea un nuevo impulso y el fin del período de una hora. Obtener las posibles secuencias de interfoliación de las instrucciones (1),(2), y (3) a partir de dicho instante, e indicar cuales de ellas son correctas y cuales incorrectas (las incorrectas son aquellas en las cuales el impulso no se contabiliza).

Respuesta

Supongamos que hay una variable entera (ficticia) llamada **OUT**, que se crea al terminar el **write** (sentencia (2)) y tiene el valor impreso (esto permite incluir en el estado del programa dicho valor impreso).

En el estado de partida, se cumple $n == k$, y a partir de ahí pueden ocurrir tres interfoliaciones posibles de las sentencias etiquetadas con los dígitos 1,2, y 3. Estas interfoliaciones son: (a) 1,2,3, (b) 2,1,3 y (c) 2,3,1.

Para cada interfoliación podemos considerar los valores de las variables en cada estado al final de cada sentencia, y podemos examinar el estado final, esto es, el valor con el que queda **n** y el valor impreso (el valor de **OUT**).

(a)

Instr.	n	OUT
	k	
n := n + 1	$k + 1$	
write (n)	$k + 1$	$k + 1$
n := 0	0	$k + 1$

(b)

Instr.	n	OUT
	k	
write (n)	k	k
n := n + 1	$k + 1$	k
n := 0	0	k

(c)

Instr.	n	OUT
	k	
write (n)	k	k
n := 0	0	k
n := n + 1	1	k

Son correctas únicamente las interfoliaciones en las cuales en el estado final se cumple:

$$\text{OUT} + n == k + 1$$

es decir, el valor impreso más el valor de contador es igual al número total de impulsos producidos desde que comenzó la hora que acaba. Evidentemente, las interfoliaciones (a) y (c) son **correctas**, mientras que la (b) es **incorrecta**.

9

Supongamos un programa concurrente en el cual hay, en memoria compartida dos vectores **a** y **b** de enteros y con tamaño par, declarados como sigue:

```
var a,b : array[0..2*n-1] of integer ; { n es una constante predefinida (>2) }
```

Queremos escribir un programa para obtener en **b** una copia ordenada del contenido de **a** (nos da igual el estado en que queda **a** después de obtener **b**).

Para ello disponemos de la función **Sort** que ordena un tramo de **a** (entre las entradas **s**, incluida, y **t**, no incluida), usando el método de la burbuja. También disponemos la función **Copiar**, que copia un tramo de **a** (desde **s**, incluido, hasta **t**, sin incluir) sobre **b** (a partir de **o**).


```

procedure Sort( s,t : integer );
  var i, j : integer ;
begin
  for i := s to t-1 do
    for j:= s+1 to t-1 do
      if a[i] < a[j] then
        swap( a[i], a[j] ) ;
      end if;
    end for;
  end for;
end

```

```

procedure Copiar( o,s,t : integer );
  var d : integer ;
begin
  for d := 0 to t-s-1 do
    b[o+d] := a[s+d] ;
  end for;
end

```

La función **swap** intercambia dos variables. El programa para ordenar se puede implementar de dos formas:

- Ordenar todo el vector **a**, de forma secuencial con la función **Sort**, y después copiar cada entrada de **a** en **b**, con la función **Copiar**.
- Ordenar las dos mitades de **a** de forma concurrente, y después mezclar dichas dos mitades en un segundo vector **b** (para mezclar usamos un procedimiento **Merge**).

A continuación vemos el código de ambas versiones:

```

procedure Secuencial() ;
  var i : integer ;
begin
  Sort( 0, 2*n );      { ordena a }
  Copiar( 0, 0, 2*n ); { copia a en b }
end

```

```

procedure Concurrente() ;
begin
  cobegin
    Sort( 0, n );
    Sort( n, 2*n );
  coend
  Merge( 0, n, 2*n );
end

```

El código de **Merge** se encarga de ir leyendo las dos mitades de **a**. En cada paso primero se selecciona el menor elemento de los dos siguientes por leer (uno en cada mitad), y después se escribe dicho menor elemento en la siguiente mitad del vector mezclado **b**. Al acabar este bucle, será necesario copiar el resto de elementos no leídos de una de las dos mitades. El código es el siguiente:

```

procedure Merge( inferior, medio, superior: integer ) ;
  var escribir : integer := 0 ;           { siguiente posicion a escribir en b      }
  var leer1    : integer := inferior ;    { siguiente pos. a leer en primera mitad de a }
  var leer2    : integer := medio      ; { siguiente pos. a leer en segunda mitad de a }
begin
  { mientras no haya terminado con alguna mitad }
  while leer1 < medio and leer2 < superior do begin
    if a[leer1] < a[leer2] then begin { minimo en la primera mitad }
      b[escribir] := a[leer1] ;
      leer1 := leer1 + 1 ;
    end else begin { minimo en la segunda mitad }
      b[escribir] := a[leer2] ;
      leer2 := leer2 + 1 ;
    end
    escribir := escribir+1 ;
  end
  { se ha terminado de copiar una de las mitades, copiar lo que quede de la otra }
  if leer2 >= superior then Copiar( escribir, leer1, medio ) ; { copiar primera }
  else Copiar( escribir, leer2, superior ) ; { copiar segunda }
end

```

Llamaremos $T_s(k)$ al tiempo que tarda el procedimiento **Sort** cuando actúa sobre un segmento del vector con k entradas. Suponemos que el tiempo que (en media) tarda cada iteración del bucle interno que hay en **Sort** es la unidad (por definición). Es evidente que ese bucle tiene $k(k-1)/2$ iteraciones, luego:

$$T_s(k) = \frac{k(k-1)}{2} = \frac{1}{2}k^2 - \frac{1}{2}k$$

El tiempo que tarda la versión secuencial sobre $2n$ elementos (llamaremos S a dicho tiempo) será el tiempo de **Sort** ($T_s(2n)$) más el tiempo de **Copiar** (que es $2n$, pues copiar un elemento tarda una unidad de tiempo), luego

$$S = T_s(2n) + 2n = \frac{1}{2}(2n)^2 - \frac{1}{2}(2n) + 2n = 2n^2 + n$$

con estas definiciones, calcula el tiempo que tardará la versión paralela, en dos casos:

- (1) Las dos instancias concurrentes de **Sort** se ejecutan en el mismo procesador (llamamos P_1 al tiempo que tarda).
- (2) Cada instancia de **Sort** se ejecuta en un procesador distinto (lo llamamos P_2)

escribe una comparación cualitativa de los tres tiempos (S, P_1 y P_2).

Para esto, hay que suponer que cuando el procedimiento **Merge** actúa sobre un vector con p entradas, tarda p unidades de tiempo en ello, lo cual es razonable teniendo en cuenta que en esas circunstancias **Merge** copia p valores desde **a** hacia **b**. Si llamamos a este tiempo $T_m(p)$, podemos escribir

$$T_m(p) = p$$

Respuesta (privada)

- (1) Sobre un procesador el coste total de la versión paralela (P_1) sería el de dos ordenaciones secuenciales de n elementos cada una, (es decir $2T_s(n)$), más el coste de la mezcla secuencial (que es $T_m(2n)$), esto es:

$$P_1 = 2T_s(n) + T_m(2n) = (n^2 - n) + 2n = n^2 + n$$

Si comparamos $P_1 = n^2 + n$ con $S = 2n^2 + n$, vemos que, aun usando un único procesador en ambos casos, para valores de n grandes la versión potencialmente paralela tarda la mitad de tiempo que la secuencial.

- (2) Sobre dos procesadores, el coste de la versión paralela (P_2) será el de la ejecución concurrente de dos versiones de **Sort** iguales sobre n elementos cada una, por tanto, será igual a $T_s(n)$. Después, la mezcla se hace en un único procesador y tarda lo mismo que antes, $T_m(2n)$, luego:

$$P_2 = T_s(n) + T_m(2n) = \left(\frac{1}{2}n^2 - \frac{1}{2}n\right) + 2n = \frac{1}{2}n^2 + \frac{3}{2}n$$

ahora vemos que (de nuevo para n grande), el tiempo P_2 es aproximadamente la mitad de P_1 , como era de esperar (ya que se usan dos procesadores), y por supuesto P_2 es aproximadamente la cuarta parte de S .

10

Supongamos que tenemos un programa con tres matrices (**a**, **b** y **c**) de valores flotantes declaradas como variables globales. La multiplicación secuencial de **a** y **b** (almacenando el resultado en **c**) se puede hacer mediante un procedimiento **MultiplicacionSec** declarado como aparece aquí:

```
var a, b, c : array[1..3,1..3] of real ;

procedure MultiplicacionSec()
  var i,j,k : integer ;
begin
  for i := 1 to 3 do
    for j := 1 to 3 do begin
      c[i,j] := 0 ;
      for k := 1 to 3 do
        c[i,j] := c[i,j] + a[i,k]*b[k,j] ;
      end
    end
  end
end
```

Escribir un programa con el mismo fin, pero que use 3 procesos concurrentes. Suponer que los elementos de las matrices **a** y **b** se pueden leer simultáneamente, así como que elementos distintos de **c** pueden escribirse simultáneamente.

Respuesta (privada)

Para implementar el programa, haremos que cada uno de esos 3 procesos concurrentes (llamados **CalcularFila**) calcule y escriba un conjunto distinto de entradas de **c**. Por simplicidad (y equidad entre los procesos), lo más conveniente es hacer que cada uno de ellos calcule una fila de **c** (o cada uno de ellos una columna)

```

var a, b, c : array [1..3,1..3] of real ;

process CalcularFila[ i : 1..3 ] ;
  var j, k : integer ;
begin
  for j := 1 to 3 do begin
    c[i,j] := 0 ;
    for k := 1 to 3 do
      c[i,j] := c[i,j] + a[i,k]*b[k,j] ;
    end
  end
end

```

11

Un trozo de programa ejecuta nueve rutinas o actividades ($P_1, P_2, \dots, P_9$), repetidas veces, de forma concurrentemente con **cobegin coend** (ver la figura de la izquierda), pero que requieren sincronizarse según determinado grafo (ver la figura de la derecha):

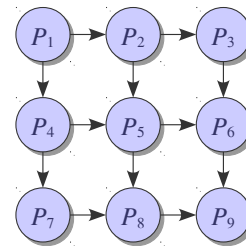
Trozo de programa:

```

while true do
cobegin
  P1 ; P2 ; P3 ;
  P4 ; P5 ; P6 ;
  P7 ; P8 ; P9 ;
coend

```

Grafo de sincronización:



Supón que queremos realizar la sincronización indicada en el grafo, usando para ello llamadas desde cada rutina a dos procedimientos (**EsperarPor** y **Acabar**). Se dan los siguientes hechos:

- El procedimiento **EsperarPor** (*i*) es llamado por una rutina cualquiera (la número *k*) para esperar a que termine la rutina número *i*, usando espera ocupada. Por tanto, se usa por la rutina *k* al inicio para esperar la terminación de las otras rutinas que corresponda según el grafo.
- El procedimiento **Acabar** (*i*) es llamado por la rutina número *i*, al final de la misma, para indicar que dicha rutina ya ha finalizado.
- Ambos procedimientos pueden acceder a variables globales en memoria compartida.
- Las rutinas se sincronizan única y exclusivamente mediante llamadas a estos procedimientos, siendo la implementación de los mismos completamente transparente para las rutinas.

Escribe una implementación de **EsperarPor** y **Acabar** (junto con la declaración e inicialización de las variables compartidas necesarias) que cumpla con los requisitos dados.

Respuesta (privada)

Una posible solución consiste en usar un vector de valores lógicos que indican si cada proceso ha terminado o no. Hay que tener en cuenta que, puesto que la ejecución concurrente de todas las rutinas está en un bucle, dicho vector debe reiniciarse entre una iteración del bucle y la siguiente. Para ello realizamos dicha reinicialización cuando el proceso 9 (el último) señale que ha acabado (en **Acabar**). La implementación queda como sigue:

```
{ compartido entre todas las tareas }
var finalizado : array [1..9] of boolean := (false, false, ..., false) ;

procedure EsperarPor( i : integer )
begin
    while not finalizado[i] do
        begin end
    end
end

procedure Acabar( i : integer )
var j : integer ;
begin
    if i < 9 then
        finalizado[i] := true ;
    else
        for j := 1 to 9 do
            finalizado[j] := false ;
        end
    end
end
```

12

En el problema anterior los procesos **P1**, **P2**, ..., **P9** se ponen en marcha usando **cobegin/coend**. Escribe un programa equivalente, que ponga en marcha todos los procesos, pero que use declaración estática de procesos, usando un vector de procesos **P**, con índices desde 1 hasta 9, ambos incluidos. El proceso **P[n]** contiene un bucle infinito, y en cada iteración se ejecuta P_n . Ejecutar P_n supone incluir las llamadas necesarias a **EsperarPor** (con la misma implementación que antes), luego una secuencia de instrucciones **Sn**, y finalmente las llamada a **Acabar**. Se incluye aquí un plantilla:

```
Process P[ n : 1..9 ]
begin
    while true do begin
        ..... { esperar (si es necesario) a los procesos que corresponda }
        Sn ;    { sentencias específicas de este proceso (desconocidas) }
        ..... { señalar que hemos terminado }
    end
end
```

Respuesta (privada)

Esta es una posible implementación, en la cual se busca la máxima simplicidad (menor número de instrucciones posibles).

El proceso de índice **n** tiene *arriba* al proceso **n-3** (*arriba* según se ha escrito el pseudo-código en el enunciado del problema anterior), pero esto solo ocurre si el proceso no está en la primera fila, es decir, solo si **n** es mayor estricto que 3). Además, el proceso **n** tiene a su izquierda al proceso de índice **n-1** (pero solo si **n** no está en la primera columna, es decir, solo si el valor **n-1** no es múltiplo de 3, equivalente a que al reducir **n-1** módulo 3, de un valor mayor que 0). Por tanto, el código es como sigue:

```
process P[ n : 1..9 ]
begin
  while true do begin
    { comprobamos qué procesos debemos esperar en función de 'n' }
    if n > 3 then EsperarPor( n-3 ); { esperamos proceso de arriba }
    if n-1 mod 3 > 0 then EsperarPor( n-1 ); { esperamos proceso de la izqda. }

    Sn ; { sentencias específicas de este proceso (desconocidas) }

    Acabar( n ); { señalar que hemos terminado }
  end
end
```